

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 13: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 17 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** Define the complexity class **PSPACE**. What resource is bounded, and what resource is left unrestricted? Explain the “recycling of space” intuition that lets a PSPACE algorithm run in exponential time but polynomial space.
- A2.** Prove that $\mathbf{NP} \subseteq \mathbf{PSPACE}$. Your argument should explain why enumerating exponentially many certificates does not blow up the *space* usage.
- A3.** State the QSAT (TQBF) problem precisely. How does it generalize SAT, and why does the recursive evaluation of a fully-quantified formula use only polynomial space despite exploring exponentially many assignments?
- A4.** Explain the precise correspondence between a quantified boolean formula $\exists x_1 \forall x_2 \exists x_3 \cdots \Phi$ and a two-player game. Which quantifier corresponds to which player, and what does the truth of the formula say about the game?
- A5.** Define the *minimax value* of a position in a two-player, zero-sum, perfect-information game. Give the recursive definition for leaf nodes, Max nodes, and Min nodes.
- A6.** State, in pseudocode or precise prose, the alpha-beta pruning algorithm. What do the parameters α and β represent, and under what condition does a node prune its remaining children?
- A7.** Prove that alpha-beta pruning returns the same value as plain minimax. (A proof sketch focusing on *why* a pruned subtree cannot affect the root value is sufficient.)
- A8.** Define what it means for a parameterized problem to be **fixed-parameter tractable (FPT)**. Contrast the running time $f(k) \cdot \text{poly}(n)$ with both a polynomial-time algorithm and a 2^n brute force, explaining when FPT is a genuine win.
- A9.** State the Vertex Cover decision problem. Prove the *branching lemma*: for any edge (u, v) , every vertex cover contains u or v . Explain how this lemma yields a bounded search tree.

- A10.** Prove that the bounded-search-tree algorithm for Vertex Cover runs in $O(2^k \cdot (n + m))$ time and correctly decides whether a cover of size $\leq k$ exists. Why is this dramatically better than the $\binom{n}{k}$ brute force when k is small?
- A11.** State and justify the two kernelization rules for Vertex Cover (the high-degree rule and the isolated-vertex rule). For the high-degree rule, prove that a vertex of degree $> k$ must lie in every cover of size $\leq k$.
- A12.** Prove the kernel-size bound for Vertex Cover: after the two reduction rules are applied exhaustively, a yes-instance has at most k^2 edges. Why does this make brute force on the kernel depend only on k (and not on the original n)?
- A13.** State the Maximum Independent Set problem on a tree, and give the dynamic program that solves it in linear time. Write the recurrences for $\text{incl}[v]$ and $\text{excl}[v]$ and explain each term.
- A14.** Prove that the tree DP for Maximum Independent Set is correct, by induction on subtree size. Explain why including a vertex v forces the use of $\text{excl}[c]$ for each child c , while excluding v uses $\max(\text{incl}[c], \text{excl}[c])$.
- A15.** Describe the general **branch-and-bound** method for an optimization problem. Define the *incumbent* and the *bound*, and state the pruning rule. For 0/1 Knapsack, explain why the *fractional* (LP-relaxation) value is a valid *upper* bound on any integral completion.
- A16.** For branch-and-bound TSP, explain what makes a lower bound *admissible* and why admissibility is required for the pruning to remain correct (i.e., to never discard the optimal tour). Give one concrete admissible lower bound.
- A17.** State the 2-SAT problem and explain the implication-graph construction. State the SCC criterion for satisfiability and explain, at a high level, why a variable x and its negation $\neg x$ lying in the same strongly connected component implies unsatisfiability.

Part B: Quiz

- B1. (True/False)** Every problem in NP is also in PSPACE.
- B2. (Multiple choice)** The canonical PSPACE-complete problem is:
- (a) 3-SAT
 - (b) Quantified SAT (QSAT / TQBF)
 - (c) Maximum Independent Set
 - (d) 2-SAT
- B3. (Multiple choice)** Compared to plain minimax on the same game tree, alpha-beta pruning:
- (a) May return a different (worse) value but runs faster
 - (b) Returns the same value and never visits more leaves
 - (c) Returns the same value but always visits exactly as many leaves
 - (d) Only works for trees of depth 2
- B4. (True/False)** A parameterized algorithm running in time $O(2^k \cdot n)$ is fixed-parameter tractable in the parameter k .
- B5. (Short answer)** In the FPT bounded-search-tree algorithm for Vertex Cover, on which two vertices does the algorithm branch, and what is the depth of the search tree?
- B6. (True/False)** In Vertex Cover kernelization, a vertex of degree greater than k must be included in every vertex cover of size at most k .
- B7. (Multiple choice)** Maximum Independent Set on a tree can be solved in:
- (a) $O(2^n)$ time only (it is NP-hard even on trees)
 - (b) $O(n)$ time by dynamic programming
 - (c) $O(n!)$ time only
 - (d) $O(n^3)$ time but not faster
- B8. (Short answer)** In branch-and-bound for 0/1 Knapsack, what quantity is used as the *upper bound* to prune a subtree, and why is it an upper bound?
- B9. (Multiple choice)** 2-SAT is satisfiable if and only if, in its implication graph:
- (a) the graph is acyclic
 - (b) no variable x has x and $\neg x$ in the same SCC
 - (c) the graph is strongly connected
 - (d) every clause has three literals
- B10. (True/False)** Iterative-deepening DFS uses memory proportional to the number of states at the current depth limit (like BFS's frontier), which is its main advantage over plain DFS.

Part C: Solutions

Part A

- A1. PSPACE** is the class of decision problems solvable by an algorithm using *working memory* (*space*) bounded by a polynomial in the input size; the *time* is unrestricted (it may be exponential). The recycling intuition: a PSPACE algorithm may examine exponentially many configurations *one at a time*, reusing the same polynomial-size memory for each and discarding it before moving to the next. Thus even though the number of configurations – and hence the running time – is exponential, the *simultaneous* memory footprint stays polynomial.
- A2. Claim: $\text{NP} \subseteq \text{PSPACE}$.** Let $L \in \text{NP}$ with polynomial-time verifier V that, given input x and a candidate certificate y (of polynomial length), checks $x \in L$. To decide x in polynomial space, enumerate all candidate certificates y in some fixed order; for each, run $V(x, y)$ (which uses polynomial space) and accept if it accepts. Crucially, we process one y at a time and *reuse* the same $O(\text{poly})$ memory for each verification, discarding y and V 's workspace before the next. So although there are exponentially many y (giving exponential time), the space used at any instant is polynomial. Hence $L \in \text{PSPACE}$.
- A3. QSAT (TQBF):** given a fully-quantified boolean formula $Q_1x_1 Q_2x_2 \cdots Q_nx_n \Phi(x_1, \dots, x_n)$ with each $Q_i \in \{\exists, \forall\}$, decide whether it is **true**. It generalizes SAT, which is the all- $\exists$ special case $\exists x_1 \cdots \exists x_n \Phi$. The recursive evaluator tries both values of the outermost variable, recursing on the remainder, and combines with “or” for $\exists$, “and” for $\forall$. The recursion stack has depth n (one frame per variable), each frame holding $O(1)$ extra state, and a leaf evaluates Φ on a complete assignment in space $O(|\Phi|)$. So although the recursion explores up to 2^n leaves over exponential time, only one root-to-leaf path exists in memory at any instant: total space $O(n + |\Phi|)$, polynomial.
- A4.** Read the quantifiers as alternating moves of two players. $\exists x_i$ is a move by the *first player* (“I get to choose x_i ”), and $\forall x_i$ is a move by the *opponent* (“whatever the opponent chooses for x_i ”). The matrix Φ is the winning condition for the first player. The formula $\exists x_1 \forall x_2 \exists x_3 \cdots \Phi$ is **true** iff the first player has a *winning strategy*: there is a choice of x_1 such that for all choices of x_2 there is a choice of $x_3, \dots$, making Φ true. This is exactly the structure of deciding the winner of a finite perfect-information game.
- A5.** The **minimax value** is defined recursively. A *leaf* has value equal to its payoff (from Max's viewpoint). A *Max node* (Max to move) has value equal to the *maximum* of its children's values. A *Min node* (Min to move) has value equal to the *minimum* of its children's values. The root's minimax value is the payoff under optimal play by both sides.
- A6.** Alpha-beta carries two bounds down the recursion: α is the best (largest) value Max can already *guarantee* along the current path, and β is the best (smallest) value Min can already guarantee. At a Max node we update $\alpha \leftarrow \max(\alpha, v)$ after each child and at a Min node we update $\beta \leftarrow \min(\beta, v)$. A node *prunes* (stops examining remaining children) as soon as $\alpha \geq \beta$: the opponent already has a choice elsewhere at least this good, so the remaining children cannot influence the outcome on this path.
- A7. Claim:** alpha-beta returns the minimax value. A subtree is pruned at a node N only after N has a child value forcing $\alpha \geq \beta$. In that situation, the ancestor of N on the opposite side (a Min node above a Max N , or vice versa) already has an alternative whose value is at least as good for it as anything N could now produce; concretely, any value the pruned subtree might

contribute would push N 's max (or min) only in a direction that the opposite-type ancestor would reject when it takes its own min (or max). Hence the pruned values can never become the chosen value on the path to the root, so the root value is identical to plain minimax. (Pruning changes only *which* leaves are examined, never the computed value.)

- A8.** A parameterized problem with input size n and parameter k is **FPT** if it is solvable in time $f(k) \cdot \text{poly}(n)$ for some function f depending only on k . The exponential cost (if any) is isolated in $f(k)$. Contrast: a polynomial-time algorithm has no $f(k)$ factor at all; a 2^n brute force is exponential in the full input size and useless for large n . FPT is a genuine win precisely when k is small (so $f(k)$ is a fixed constant) while n is large – then $f(k) \cdot \text{poly}(n)$ is effectively polynomial, unlike 2^n which depends on the (large) n .
- A9. Vertex Cover decision:** given $G = (V, E)$ and integer k , is there $C \subseteq V$ with $|C| \leq k$ such that every edge has an endpoint in C ? **Branching lemma:** for any edge (u, v) , any vertex cover C must contain u or v , because the edge (u, v) must be covered and its only endpoints are u and v . **Bounded search tree:** pick any uncovered edge (u, v) ; recurse with u added to the cover (budget $k - 1$) and, if that fails, with v added. Each branch reduces the budget by one, so the search tree has depth $\leq k$ and branching factor 2.
- A10. Correctness:** the recursion enumerates, for each edge it encounters, both choices of endpoint to cover it; by the branching lemma every cover is consistent with some such sequence of choices, so a cover of size $\leq k$ (if any) is found at a leaf of depth $\leq k$; conversely any returned set covers all edges (we only return success when no edges remain) and has size $\leq k$ (one vertex added per level, k levels). **Running time:** the tree has depth $\leq k$ and branching factor 2, so $\leq 2^k$ leaves and $O(2^k)$ nodes; each node spends $O(n + m)$ filtering incident edges, giving $O(2^k(n + m))$. This beats the $\binom{n}{k} = \Theta(n^k)$ brute force because the exponential factor depends only on k , not on n : for small fixed k and large n , $2^k(n + m)$ is essentially linear in the input.
- A11. High-degree rule:** if a vertex v has degree $> k$, put v in the cover, delete v and its incident edges, and decrement k . *Proof it is forced:* suppose a cover C of size $\leq k$ omits v . Then every one of v 's $> k$ incident edges must be covered by its *other* endpoint, and these other endpoints are distinct vertices, all in C ; that is $> k$ distinct vertices in C , contradicting $|C| \leq k$. **Isolated-vertex rule:** a degree-0 vertex covers no edge, so removing it changes no edge's coverage and never increases the required cover size; drop it. Both rules preserve the set of yes/no answers (with k adjusted by the high-degree rule).
- A12.** After the rules are applied exhaustively, every remaining vertex has degree $\leq k$ (else the high-degree rule would fire). In a yes-instance there is a cover C with $|C| \leq k$. Every edge has an endpoint in C , and each vertex of C has degree $\leq k$, so it covers at most k edges; hence the total number of edges is at most $|C| \cdot k \leq k \cdot k = k^2$. The non-isolated vertices all sit on these $\leq k^2$ edges, so there are $O(k^2)$ of them. The original n has dropped out entirely: a brute-force search on the kernel costs a function of k only (e.g. $2^{O(k^2)}$) or, combined with the search tree, $2^k \cdot \text{poly}(k)$, independent of n .
- A13. Maximum Independent Set on a tree:** find the largest S of pairwise non-adjacent vertices. Root the tree anywhere; in post-order compute

$$\text{incl}[v] = 1 + \sum_{c \in \text{children}(v)} \text{excl}[c], \quad \text{excl}[v] = \sum_{c \in \text{children}(v)} \max(\text{incl}[c], \text{excl}[c]).$$

$\text{incl}[v]$ is the best independent set of v 's subtree that *includes* v : the “1” counts v itself, and since v is in the set, no child may be (so each child contributes $\text{excl}[c]$). $\text{excl}[v]$ is the best that *excludes* v : each child is free, so it contributes the better of including or excluding it, $\max(\text{incl}[c], \text{excl}[c])$. The answer is $\max(\text{incl}[\text{root}], \text{excl}[\text{root}])$.

- A14. Correctness (induction on subtree size).** *Base:* a leaf v has $\text{incl}[v] = 1$ (just v) and $\text{excl}[v] = 0$ (empty), both correct. *Step:* assume the values are correct for all children of v . Any independent set of v 's subtree either includes v or not. If it includes v , then no child of v may be included (the edge $v-c$ would be violated), so the best such set is v plus, independently in each child's subtree, the best set *excluding* that child – exactly $1 + \sum_c \text{excl}[c] = \text{incl}[v]$. If it excludes v , each child's subtree may use its own best set, including or excluding the child freely – exactly $\sum_c \max(\dots) = \text{excl}[v]$. Taking the max at the root covers both root cases. **Time:** each vertex is processed once with work proportional to its number of children; the children counts sum to $n - 1$, so total $O(n)$.
- A15. Branch-and-bound** explores a search tree of partial solutions but prunes using bounds. The *incumbent* is the best complete solution found so far. The *bound* at a node is an optimistic estimate of the best objective achievable by any completion of that node's partial solution (an *upper* bound for maximization, *lower* bound for minimization). **Pruning rule:** if a node's bound cannot beat the incumbent (e.g. for maximization, $\text{bound} \leq \text{incumbent}$), discard the entire subtree. *0/1 Knapsack upper bound:* the fractional (LP-relaxation) optimum – greedily take whole items by value/weight ratio, then a fraction of the next to fill capacity – is achievable by a *relaxed* problem that allows fractional items. Since the integral 0/1 problem is more constrained, its optimum cannot exceed the fractional one; hence the fractional value is a valid upper bound.
- A16.** A lower bound is **admissible** if it never *overestimates* the true minimal cost to complete the partial tour. Admissibility is required for correctness: branch-and-bound prunes a node when cost so far + lower bound $\geq$ incumbent. If the bound were an overestimate, it might exceed the incumbent even though the true best completion is cheaper – causing us to prune away (and lose) the optimal tour. With an admissible (never over-estimating) bound, a pruned node provably cannot contain a tour better than the incumbent. *One admissible bound:* cost so far, plus for the current city and each unvisited city the cheapest edge that could still leave it (toward an unvisited city or back to the start). Each such cheapest edge is a lower bound on the cost charged to that city by any completion, so their sum cannot exceed the true completion cost.
- A17. 2-SAT:** satisfiability of a CNF formula where every clause has exactly two literals. **Implication graph:** create two nodes per variable (x and $\neg x$); for each clause $(a \vee b)$, add directed edges $\neg a \rightarrow b$ and $\neg b \rightarrow a$ (each is logically equivalent to the clause). **SCC criterion:** the formula is satisfiable iff no variable x has x and $\neg x$ in the same strongly connected component. *Why:* implications are transitive, and all literals in one SCC are mutually reachable, so they are forced to share the same truth value. If x and $\neg x$ are in the same SCC, they would be forced equal – impossible, since a variable and its negation must differ. Conversely, if no such collision occurs, assigning truth values in reverse topological order of the SCCs yields a consistent satisfying assignment.

Part B

- B1. True. $\text{NP} \subseteq \text{PSPACE}$:** certificates can be enumerated one at a time, reusing polynomial space for each verification (see A2).
- B2. (b) Quantified SAT (QSAT / TQBF).** The alternating $\exists/\forall$ quantifiers make it the canonical PSPACE-complete problem; 3-SAT is NP-complete and 2-SAT is in P.
- B3. (b) Returns the same value and never visits more leaves.** Alpha-beta is value-preserving (A7); pruning can only *remove* leaf visits, never add them.
- B4. True.** $O(2^k \cdot n)$ has the form $f(k) \cdot \text{poly}(n)$ with $f(k) = 2^k$, the definition of FPT.
- B5.** It branches on the **two endpoints u and v of an (uncovered) edge**; the search tree has **depth $\leq k$** (one vertex added per level).
- B6. True.** If such a vertex were omitted, its $> k$ edges would need $> k$ distinct other endpoints in the cover, exceeding the budget k (see A11).
- B7. (b) $O(n)$ time by dynamic programming.** The root-to-leaf include/exclude DP solves it in linear time on trees, even though the problem is NP-hard on general graphs.
- B8.** The **fractional (LP-relaxation) optimum** of the knapsack. It is an upper bound because allowing fractional items can only *increase* the achievable value relative to the integral 0/1 problem, so no integral completion can exceed it.
- B9. (b) no variable x has x and $\neg x$ in the same SCC.** This is the Aspvall–Plass–Tarjan criterion.
- B10. False.** It is the *opposite*: iterative-deepening DFS uses memory proportional to the current depth limit ($O(\ell)$, like DFS), *not* proportional to the frontier (like BFS) – that small memory footprint is exactly its advantage, while still finding a shortest-in-moves solution.